# The `csv` module

**Status:** not covered by Software Lab-1 practicals **Where it is used:** `backend/store.py` (ensure_csv, read_all, append_row, update_row, delete_row), `backend/triage.py` (load_diseases), `scripts/trim_seed_data.py` (trim), `scripts/verify_seed_data.py` (load) **Closest lab concept:** Practical 4, functions and files — you already read CSVs by hand with `open()`, `readlines()` and `split()`; `csv` is that loop done correctly

---

## 1. What it is

`csv` is a stdlib module — no install — for reading and writing comma-separated values files. It knows the quoting rules: a field may be wrapped in double quotes, and inside those quotes a comma, a quote or even a newline is data, not structure.

It is the Lab 4 file work with one step replaced. Where you wrote:

```
with open(path, "r") as f:
    lines = f.readlines()
header = lines[0].strip().split(",")
for line in lines[1:]:
    values = line.strip().split(",")
    row = dict(zip(header, values))
```

`csv` produces the same `row` dict without the `split` — and without its failure modes, which are not hypothetical (section 4).

---

## 2. Why the project needed it

Meridian keeps its five registries as CSVs under `data/` — `patients.csv`, `visits.csv`, `medicines.csv`, `stores.csv`, `diseases.csv`, listed once in `store.py:44-50`:

```
RECORDS = {
    "patients": ("patients.csv", PATIENT_FIELDS, "patient_id"),
    "visits": ("visits.csv", VISIT_FIELDS, "visit_id"),
    "medicines": ("medicines.csv", MEDICINE_FIELDS, "medicine_id"),
    "stores": ("stores.csv", STORE_FIELDS, "store_id"),
    "diseases": ("diseases.csv", DISEASE_FIELDS, "disease_id"),
}
```

Every API call that creates, lists, updates or deletes a record ends in one of those files, so CSV parsing sits on every code path.

This is not a clean-data problem. The seeded `data/patients.csv` already contains a quoted comma, in the very first data row:

```
patient_id,name,age,gender,phone,blood_group,area,city,registered_on,notes
P-0001,Aarav Sharma,34,male,9845012345,O+,"Koramangala, 5th Block",Bengaluru,2026-01-12,Hypertension foll
```

`Koramangala, 5th Block` is one value. `split(",")` cannot know that (section 4). Spec section 9 also requires that a malformed row is reported, never silently dropped — so the reader must know precisely what a well-formed row looks like, which means real parsing.

---

## 3. The three pieces actually used

### 3.1 `csv.DictReader(f)` — iterating yields one dict per row

`store.read_all`, `store.py:102-103`:

```
    with open(path, "r", newline="") as f:
```

```
        reader = csv.DictReader(f)
```

`DictReader` consumes the first line as the header, then yields one dict per row mapping column name to cell value. On the real row above, that dict is:

```
{'patient_id': 'P-0001', 'name': 'Aarav Sharma', 'age': '34', 'gender': 'male',
 'phone': '9845012345', 'blood_group': 'O+', 'area': 'Koramangala, 5th Block',
 'city': 'Bengaluru', 'registered_on': '2026-01-12', 'notes': 'Hypertension follow-up'}
```

Note `area` survived its comma intact. Two properties to internalise:

- **Every value is a string.** `age` is `'34'`, not `34` — same as `readlines()`, you convert yourself. `registry.validate_record` (`registry.py:208`) does the type work on the way in.
- **No header skip, no slicing.** There is no `lines[0]`, no `lines[1:]`, no `zip(header, values)`. Blank lines are skipped, so a stray empty line does not become a garbage row.

`read_all` then normalises each cell (`store.py:124`):

```
            records.append({k: (row[k] or "").strip() for k in fields})
```

`backend/triage.py:78-79` uses the same reader for `diseases.csv`:

```
    with open(path, "r", newline="") as f:
        for row in csv.DictReader(f):
```

### 3.2 `csv.DictWriter(f, fieldnames=[...])`, `writeheader()`, `writerow(dict)`

`DictWriter` needs the column order handed to it; it cannot infer it. That is why `store.py` keeps the schemas as module-level constants (`store.py:19-22`):

```
PATIENT_FIELDS = [
    "patient_id", "name", "age", "gender", "phone", "blood_group",
    "area", "city", "registered_on", "notes",
]
```

Create a file containing only a header — `ensure_csv`, `store.py:72-74`:

```
    try:
        with open(path, "x", newline="") as f:
            csv.DictWriter(f, fieldnames=fields).writeheader()
```

Append one row — `append_row`, `store.py:152-155`:

```
    with open(path, "a", newline="") as f:
        if needs_newline:
            f.write("\n")
        csv.DictWriter(f, fieldnames=fields).writerow(clean)
```

Rewrite the whole file — `update_row` and `delete_row` both open mode `"w"`, write the header, then every kept row (`store.py:227-231`):

```
    with open(path, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=fields)
        writer.writeheader()
        for record in records:
            writer.writerow(record)
```

CSV has no update and no delete mode, so both operations are read-modify-rewrite — the Lab 4 merge-and-overwrite pattern. `writerow` also **quotes** any value containing a comma, which is precisely what keeps `Koramangala, 5th Block` from corrupting the file on the way out.

### 3.3 `newline=""` on `open()` — required, not stylistic

Every `open()` that feeds or receives a csv object here passes `newline=""` — `store.py:73, 102, 152, 227, 247`, `triage.py:78`, `trim_seed_data.py:33, 51`, `verify_seed_data.py:32`.

In text mode Python translates line endings on the way through: `\n` becomes `os.linesep` when writing, `\r\n` becomes `\n` when reading. The csv module does its own line-ending handling and needs the raw bytes. Without `newline=""`, on Windows the writer's `\r\n` row terminators come out as `\r\r\n`, and reading that file back in another program yields a blank line after every row — a file that looks correct in the program that wrote it and corrupted in the next. Forget the argument and `csv.reader` may even refuse the file outright with `_csv.Error: new-line character seen in unquoted field`.

---

## 4. Why `split(",")` is not a CSV parser

The Lab 4 trap, made concrete with a real row from `data/patients.csv`:

```
>>> line = 'P-0001,Aarav Sharma,34,male,9845012345,O+,"Koramangala, 5th Block",Bengaluru,2026-01-12,Hyper
>>> len(line.split(","))
11
```

The header has 10 columns. `split` produced 11, and the damage is cumulative — every field after the embedded comma is shifted one place left, with the quote characters left attached:

```
>>> line.split(",")[6]
'"Koramangala'
>>> line.split(",")[7]
' 5th Block"'
```

`dict(zip(header, values))` would bind `area='"Koramangala'`, `city="' 5th Block\""`, `city→'Bengaluru'`, and so on to the end. The patient's address becomes the city and the notes are lost.

`csv.reader` returns 10 fields for the same line, and `DictReader` returns `area == 'Koramangala, 5th Block'`.

Three things `split(",")` structurally cannot see:

- **Quoted commas** — the row above, in this project's own seed data.
- **Quoted newlines** — a `notes` cell may contain a line break:

`P-0002,Diya Patel,28,female,9845012346,A+,"Jayanagar, 4th Block",Bengaluru,2026-01-18,"Follow-up needed in June"`

`for line in f:` yields two physical lines for one record, and the second parses as a bogus row of its own. `csv` treats the newline inside quotes as part of the field. - **Escaped quotes** — a field containing `"` is written as `""` inside a quoted field, so quote counting becomes real work.

Worse, `split` fails *silently*: no exception, just plausible-looking shifted data. That is the whole argument for using the module.

---

## 5. Gotchas this project actually hit

### 5.1 The `None` key: how `read_all` detects malformed rows

When a row has **more** fields than the header, `DictReader` has no column name to attach the extras to, so it files them under the key `None`. `store.read_all` uses exactly that as its malformed-row signal (`store.py:106-114`):

```
        for row in reader:
            line += 1

            if None in row:
                skipped.append({
                    "line": line,
                    "reason": f"too many fields, expected {len(fields)}",
```

```
                })
                continue
```

The value under `None` is a **list**, because there may be several extra fields:

```
>>> next(csv.DictReader(io.StringIO('patient_id,name,age\nP-1,Aarav,34,x,y\n')))
{'patient_id': 'P-1', 'name': 'Aarav', 'age': '34', None: ['x', 'y']}
```

A stray comma left by someone hand-editing the file in a spreadsheet produces one extra field, which lands under `None`, which turns the row into a visible skip instead of a silent corruption — spec section 9. `read_all` returns `(records, skipped)` for exactly this reason.

### 5.2 A short row is *not* skipped — it is silently blanked

This is the counter-intuitive half, and it is easy to misread from the code. A row with **fewer** fields than the header is *padded*, not flagged: `DictReader` fills the absent columns with `None`, so every column key is still present. Verified against the real field list:

```
>>> row = next(csv.DictReader(io.StringIO('P-0009,Diya Patel,28')))
>>> [k for k in row if k not in PATIENT_FIELDS]
[]
>>> None in row
False
```

So `store.py:116-122`'s missing-column check does not fire, the row is accepted, and `store.py:124`'s `(row[k] or "")` turns each padded `None` into an empty string. A truncated row becomes a record full of blanks, not an error.

That missing-column branch is not dead code — it catches a *header* that disagrees with the declared `fields` list, which is what happens if someone edits the header row:

```
>>> row = next(csv.DictReader(io.StringIO('patient_id,name\nP-1,Aarav\n')))
>>> [k for k in ['patient_id', 'name', 'age'] if k not in row]
['age']
```

Because of this, `store.read_all` reports *structural* damage only. Real field validation happens earlier and separately, in `registry.validate_record` (`registry.py:208`), which is why that function exists rather than `read_all` doing the checking itself.

### 5.3 The trailing-newline trap in `append_row`

CSV has no record delimiter of its own — a row boundary *is* a newline. So if the file's last line has no terminator, an appended row is glued straight onto it. Demonstrated on a two-column file:

```
guarded   ->  [['id', 'name'], ['P-1', 'Aarav'], ['P-2', 'Diya']]
unguarded ->  [['id', 'name'], ['P-1', 'AaravP-2', 'Diya']]
```

`P-1` and `P-2` fuse into one record named `AaravP-2`. Both ids are now wrong, one patient has vanished, and `read_all` raises no error because the field count is still correct.

`append_row` therefore inspects the last byte before writing (`store.py:143-147`):

```
    needs_newline = os.path.getsize(path) > 0
    if needs_newline:
        with open(path, "rb") as f:
            f.seek(-1, 2)                    # seek to the last byte
            needs_newline = f.read(1) != b"\n"
```

Reading it line by line:

- `os.path.getsize(path) > 0` — an empty (or brand new) file has no last byte to inspect, so `seek` would fail. Guard first.

- `open(path, "rb")` — **binary** mode, so no line-ending translation can corrupt the comparison.
- `f.seek(-1, 2)` — go back one byte from the **end**. The `2` is `os.SEEK_END`; the equivalent spelled-out form is `f.seek(-1, os.SEEK_END)`. This is the Lab 4 file-position concept applied to a one-byte read.
- `f.read(1) != b"\n"` — one byte comes back as `bytes`, so it is compared against `b"\n"`, not `"\n"`. Mix that up and the comparison is always true, the guard fires on every append, and you get a blank line between rows instead of corrupted data.

The result is only ever `"yes, add a newline first"` — which is why `store.py:153-154` writes it before the row rather than after.

---

## 6. `newline=""` when writing, not only reading

Worth stating separately, because the read-side benefit (no spurious blank lines on re-read) is the one usually remembered, and the write-side cost is the one that produces corrupt files.

`csv.DictWriter` emits `\r\n` as the row terminator. Text-mode `open` *also* translates `\n` into `os.linesep` on the way out. Compose the two on Windows and each terminator is doubled before it reaches the disk. The file still opens, and it still looks right in a spreadsheet — the damage only surfaces when another tool reads it strictly, and then as doubled blank lines.

The same argument covers the other direction: a `DictReader` must be able to see `\r\n` exactly as written, because a `\r\n` sequence is what distinguishes "end of record" from "a newline that happens to sit inside a quoted field" (section 4, second bullet). Translate it away and that distinction is gone. `newline=""` disables translation in both directions and leaves the job to the csv module, which handles both conventions correctly. All eight csv `open()` calls in this project pass it; that consistency is deliberate, not cargo-culted.

---

## 7. Questions this should answer

- Why does `line.split(",")` break on `"Koramangala, 5th Block"`, and what does `csv.DictReader` return for that same field instead?
- What does one row from `csv.DictReader(f)` look like, and why is `age` the string `'34'` rather than the number `34`?
- Why must `csv.DictWriter` be given `fieldnames=` when `csv.DictReader` reads the names for itself?
- What goes wrong if you forget `newline=""` on a csv `open()`, in each of reading and writing?
- What does a `None` key in a `DictReader` row mean, and why does a row that is too *short* not produce one?
- Why does `append_row` open the file in `"rb"` and call `f.seek(-1, 2)` before writing a row?